

Inventory & Stock Management (Spring Boot)

Phase 1: Setup & Entity Creation

Ticket 1: Project Setup

Description: Create Spring Boot project with dependencies (Spring Web, Spring Data JPA, MySQL, Spring Security, JWT).

Ticket 2: Create User & Role Entities

Description: Create User entity with fields (id, name, email, password, role, mobileNo, address, status, createdAt, updatedAt). Create Role entity for role management (Admin, Dealer, Customer).

Ticket 3: Create Product Entity

Description: Create Product entity with 10+ fields (id, name, category, brand, description, price, quantity, minStockLevel, createdAt, updatedAt, dealerId).

Ticket 4: Create DTO & Mapper for Product and User

Description: Create DTO classes for ProductDTO, UserDTO. Implement mapper class to convert between Entity and DTO.

Phase 2: Security & Authentication

Ticket 5: Implement User Registration API

Description: Create API to register new users with role (Customer, Admin, Dealer). Save details in DB.

Ticket 6: Implement User Login with JWT

Description: Create API for login, validate user credentials, and return JWT token.

Ticket 7: Role-Based Access Setup

Description: Implement method-level security using Spring Security so Admin can manage products, Dealer can update stock, and Customer can view products.

Phase 3: Product CRUD (with Layers)

Ticket 8: Add Product API (Admin Only)

Description: Create POST /products API to add new product. Implement Controller → Service → ServiceImpl → Repository using DTO & Mapper.

Ticket 9: Get Product by ID API (All Roles)

Description: Create GET /products/{id} API to fetch product by ID.

Ticket 10: Get All Products API (All Roles)

Description: Create GET /products API with pagination & filtering (category, brand, price range).

Ticket 11: Update Product API (Admin Only)

Description: Create PUT /products/{id} API to update product details (name, description, price, category).

Ticket 12: Delete Product API (Admin Only)

Description: Create DELETE /products/{id} API to remove product from DB.

Phase 4: Stock Management

Ticket 13: Update Stock API (Dealer Only)

Description: Create PATCH /products/{id}/stock API to increase or decrease stock quantity.

Ticket 14: Low Stock Alert API (Admin Only)

Description: Create API to fetch products with stock quantity below minStockLevel.

Ticket 15: Transaction Log Entity & API

Description: Create TransactionLog entity with fields (id, productId, userId, changeType, quantityChanged, createdAt). Save each stock update in logs.

Phase 5: Exception Handling & Standard Response

Ticket 16: Global Exception Handling

Description: Implement @ControllerAdvice for handling exceptions (ResourceNotFound, InvalidInput, AuthenticationFailure, AccessDenied).

Ticket 17: Standard Response Wrapper

Description: Create BaseResponseDTO with fields (status, message, data, timestamp). Ensure all APIs return success/error responses in this format.

Phase 6: Testing & Finalization

Ticket 18: Unit Tests for Services

Description: Write JUnit tests for Service layer (ProductService, UserService).

Ticket 19: Integration Tests for APIs

Description: Test all endpoints with real DB + Postman collection.

Ticket 20: Documentation (Swagger Setup)

Description: Integrate Swagger UI for API documentation.

👉 With this flow:

- Freshers learn **Entity → DTO → Mapper → Repository → Service → Controller** structure.
- They also handle **Role-based security, JWT, global exceptions, and proper API responses**.
- Each ticket is **independent & clear** enough to assign separately.